```
  # ...
end
```

Specifying a layout of `nil` turns off layouts for a controller.

Sometimes you need to change the appearance of a set of pages at runtime. For example, a blogging site might offer a different-looking side menu if the user is logged in, or a store site might have different-looking pages if the site is down for maintenance. Rails supports this need with dynamic layouts. If the parameter to the `layout` declaration is a symbol, it's taken to be the name of a controller instance method that returns the name of the layout to be used:

```
class StoreController < ApplicationController

  layout :determine_layout
  # ...
  private

  def determine_layout
    if Store.is_closed?
      "store_down"
    else
      "standard"
    end
  end
end
```

Subclasses of a controller use the parent's layout unless they override it using the `layout` directive. Finally, individual actions can choose to render using a specific layout (or with no layout at all) by passing `render` the `:layout` option:

```
def rss
  render(layout: false)   # never use a layout
end
def checkout
  render(layout: "layouts/simple")
end
```